What is an operating system?

- The interface between a user's program and the hardware
- Provides a variety of services and supervisory functions:
  - Handling basic I/O
  - Allocating storage and memory
  - Protected sharing of the computer among multiple applications simultaneously
- Examples include Windows, Linux, MacOS

Distributed systems and the OS

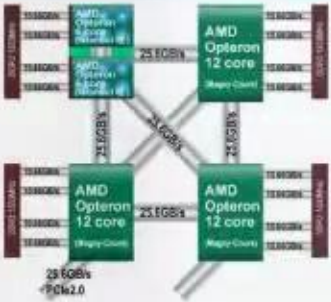
- A distributed system comprises computers linked by a network that coordinate in some way
- An OS runs on each computer, and either:
  - each OS is independent and the distributed systems software gives the impression of a "single computer" to the application; or
  - the OS itself is distributed and gives the impression of a single computer to the higher level software.

Network operating systems

- Not the same as distributed operating systems
  - Each computer has a private operating system
  - Users work at their own computer and remote login to other computers
  - Users explicitly copy files from one place to another
  - Users explicitly run programs locally or remotely
  - No fault tolerance
  - Local security mechanisms

Architectures

- Tightly coupled
  - Multiple CPUs connected at the bus level, or even on the same chip
  - Common address space
  - Eg mainframes, multicore
- Loosely coupled
  - Multiple autonomous systems connected by a network
  - Eg clusters





Summary of this talk

- Distributed operating systems on loosely-coupled systems are rare
  - This talk will describe some research efforts
- Operating systems for tightly-coupled multiprocessor systems are commonplace
  - And it turns out share many of the same design issues and performance trade-offs as distributed systems
  - It may be instructive to examine these issues and trade-offs in a different context and at a different scale

Outline

- Introduction
  - Classification of computer systems
  - OS design issues
- A selection of multiprocessor and distributed OSes
  - C.mmp/Hydra
  - C*/Medusa
  - Firefly/Taos
  - Mach
  - Amoeba
  - VAXclusters
  - K42

Taxonomies of parallel hardware

- The most well known is Flynn's taxonomy (1966)
  - Very simple
  - Many extensions and refinements subsequently
- Why did anyone care?

Taxonomies of parallel hardware

- The most well known is Flynn's taxonomy (1966)
  - Very simple
  - Many extensions and refinements subsequently
- Why did anyone care?

Back in the old days...

- Hardware and software were typically designed and built together
  - Leading to a great diversity of architectures, and hence of operating system structure
- In the last 20 years we have become accustomed to a fairly standard architecture (excluding supercomputers)
  - And because of Moore's law, and various software optimizations, computers have just kept getting faster without the need to change

Flynn's taxonomy (1966)

- Number of instruction streams, number of data streams

	Single instruction	Multiple instruction
Single data	SISD	MISD
Multiple data	SIMD	MIMD

Flynn's taxonomy: SISD

- Single instruction stream, single data stream
  - Conventional uniprocessor

Flynn's taxonomy: MIMD

- Multiple instruction streams, multiple data streams
  - Conventional multiprocessor
- Normally programmed as Single Program Multiple Data (SPMD)
- Exploits thread-level parallelism

Flynn's taxonomy: SIMD

- Single instruction stream, multiple data streams
  - Vector processor, eg GPU, x86 multimedia extensions
- A single instruction operates on many data items at once using many functional units
- Parallel execution units are synchronized
- Exploits data-level parallelism

Flynn's taxonomy: MISD

- Multiple instruction streams, single data stream
  - ?

Extended taxonomy [Johnson88]

"... we find that both the communication/synchronization mechanism and the memory structure employed in an architecture may have significant effects on the performance, and on the programmer's view of that architecture."

	Shared Variables	Message Passing
Global Memory	GMSV	GMMP
Distributed Memory	DMSV	DMMP

Extended taxonomy (cont)

- ~~Global~~ Centralized memory
  - Memory modules equally accessible from any processor
- Distributed memory
  - Different access to memory modules depending on the processor (either in terms of latency, or access rights)
- Shared ~~variables~~ memory
  - Communication and synchronization take place implicitly through loads and stores
- Message passing
  - Communication and synchronization take place through exchanging messages

DMSV: Distributed and shared memory

- Non-uniform memory access (NUMA)
- Single address space (as for GMSV)
- Can have lower latency for nearby memory
- Potential to scale to larger sizes than UMA
- Additional concern is that performance also depends on code and data placement
  - More complicated to program

DMSV: Distributed and shared memory

- Non-uniform memory access (NUMA)
- Single address space (as for GMSV)
- Can have lower latency for nearby memory
- Potential to scale to larger sizes than UMA
- Additional concern is that performance also depends on code and data placement
  - More complicated to program

Outline

- Introduction
  - Classification of computer systems
  - OS design issues
- A selection of multiprocessor and distributed OSes
  - C.mmp/Hydra
  - C*/Medusa
  - Firefly/Taos
  - Mach
  - Amoeba
  - VAXclusters
  - K42

Design issues

- This talk will focus on the following issues for a selection of historical systems:
  - Shared memory vs message passing
  - Replication/caching
  - Exploiting parallelism
  - Performance debugging
- Lots of other important aspects too:
  - *Reliability, correctness, ease of programming, scaling, power, cost, ...*

Shared memory vs message passing

- What style of communication is used inter-process (different address spaces) and intra-process (between threads in the same address space)?
- Implications for
  - Synchronization mechanisms
  - Modularity (separating interface from implementation)
  - Granularity of protection domains
  - Tracing and debugging
  - The performance costs of:
    - exchanging large amounts of data
    - exchanging small amounts of data

Replication/caching

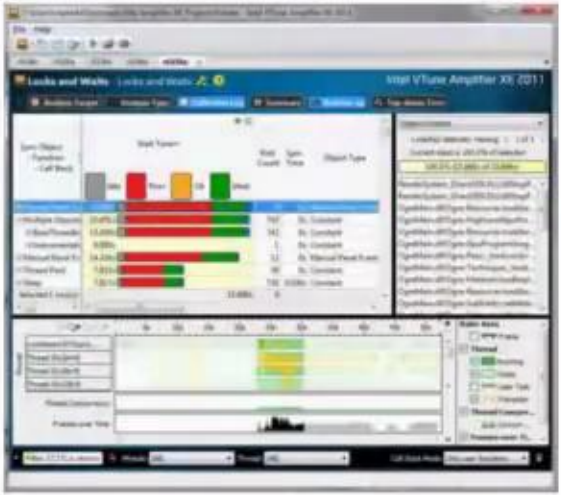
- Cached data can be private (only one copy, stored close to the owning processor) or shared (may be replicated in many caches)
- Benefits
  - Reduced access latency
  - Reduced demands on memory bandwidth
  - Reduced contention on read-only data
- Drawbacks of caching shared data
  - Need cache coherence protocol

Exploiting parallelism

- How easy is it for the programmer to make use of the available parallelism of the machine?
- What support does the OS offer?

Performance debugging

- What mechanisms are available for inspecting performance and analyzing problems across multiple processors?



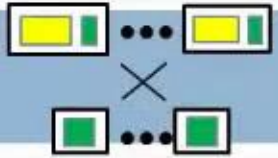
Multiprocessor OSes

- Multiprocessor computers were anticipated by the research community long before they became mainstream
- But relatively few operating systems have been designed from the outset for multiprocessor hardware
- A multiprocessor OS:
  - Runs on a tightly-coupled (ie shared-memory) multiprocessor machine
  - Provides system-wide OS abstractions
    - Global namespace
    - Global resource management
    - "Single system image"

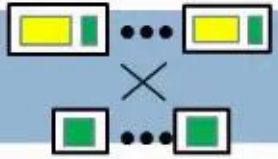
Examples (mostly research)

- Multics (MIT, Bell Labs, GE, 1965-mid 1980s)
- Hydra (CMU, early 1970s)
- Medusa (CMU, late 1970s)
- Taos (DEC SRC, mid 1980s)
- Mach (CMU, 1985-94)
- Hive (Stanford, early 1990s)
- K42 (IBM Research, 1997-2006)

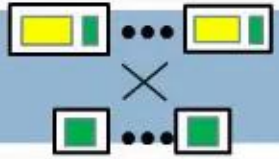
C.mmp and Hydra



C.mmp and Hydra



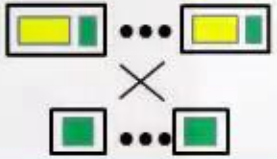
C.mmp and Hydra

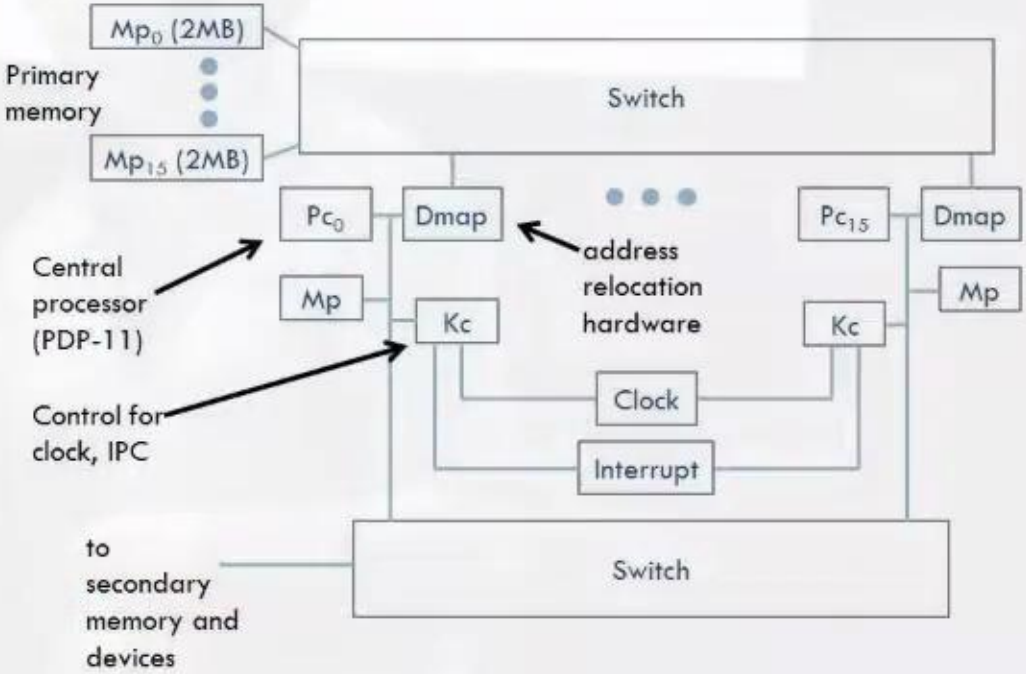


Hydra

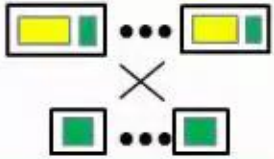
- Early 1970s
- Multiprocessor operating system for C.mmp (Carnegie-Mellon Multi-Mini-Processor)
  - Up to 16 PDP-11 processors
  - Up to 32MB memory
- Design goals:
  - Effective utilization of hardware resources
  - Base for further research into OSes and runtimes for multiprocessor systems

C.mmp multiprocessor



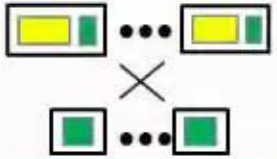


C.mmp



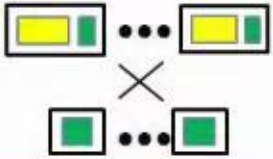
- Crossbar switch to access memory banks
  - Up to min(m,p) simultaneous conversations between m primary memories and p processors
  - Uniform memory access (~1us if no contention)
  - But contention was a big concern
- No caching of shared data
  - 8kb private memory on processors used by the OS
- C.mmp in 1978:
  - 16 PDP-11 processors (2.5-5us/instruction)
  - 3MB shared memory
  - Cost roughly $600,000 for processors, memory and switch

Hydra



- General purpose OS designed from the outset to be distributed
  - Symmetric
    - No master-slave relationships
    - Processes can be run on any processor
- Object-oriented
- Capability based
- Separation of policy and mechanism
  - eg scheduling policy defined at user level

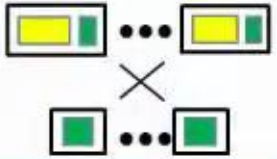
Debugging



- Hardware Monitor
  - Device to monitor signals on a PDP-11's bus
  - Useful for measuring activity of a single processor
- Kernel Tracer
  - Built into the kernel
  - Record selected OS events (eg synchronization, context swaps)

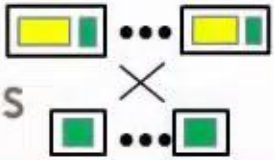
"The importance of these tools should not be underestimated. In any system as complex as an operating system, design decisions are often based on intuitive assumptions of performance tradeoffs."

Design issues



- Shared memory vs message passing
  - Centralized, shared memory (single address space)
- No caching
  - Apart from a small, private cache local to each processor
- Exploiting parallelism
  - Symmetric: any process can run on any processor(s)
  - Kernel provides locks and semaphores
  - Locking on data structures not code
    - Standard now, innovative at the time!
- Performance debugging
  - Built-in support

Discussion: the lack of caches



"Caches present a potential difficulty for multiprocessor systems because data shared between processors may be modified in one processor's cache without the modification being reflected to other processors. We chose to solve this problem by avoiding it..."

and

"The memory contention that has plagued other multiprocessors was simply not present under normal loads."

Why did the lack of caches not matter?

- Processors were slow relative to memory
  - Collective processor speed ~6 million instructions per second
  - Total memory bandwidth ~500 million bits per second
- Magnetic core memory was expensive and caches were small
  - Performance gain per buck not as great as today
- Virtual address space 64K vs physical memory 3MB!
  - If accessing more than 64K of data, the time to make individual memory accesses was dwarfed by the time to reload memory bank registers
- Contention reduced by a clever implementation of mutexes using a feature of the hardware that avoided the need to spin on shared memory accesses

C* and Medusa



Cm*

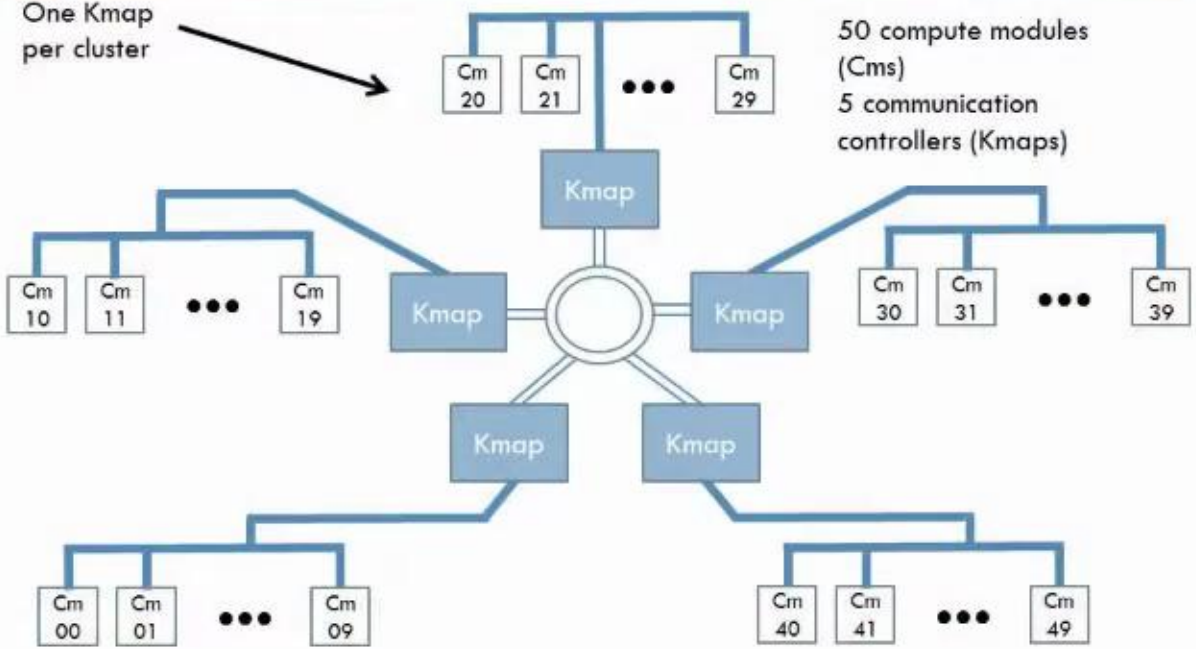
- CMU, started 1975
- Replaced central cross-bar switch of C.mmp with a distributed, bus-oriented interconnection scheme between processor-memory pairs
- Goal was to improve scalability over C.mmp

Cm*

- 50 processors, 3MB shared memory
- Each processor is a DEC LSI-11 processor with bus, local memory and peripherals
- Set of clusters (up to 14 processors per cluster) connected by a bus
- Memory can be accessed locally, within the cluster and at another cluster
- No caches
- 2 operating systems were developed: StarOS and Medusa

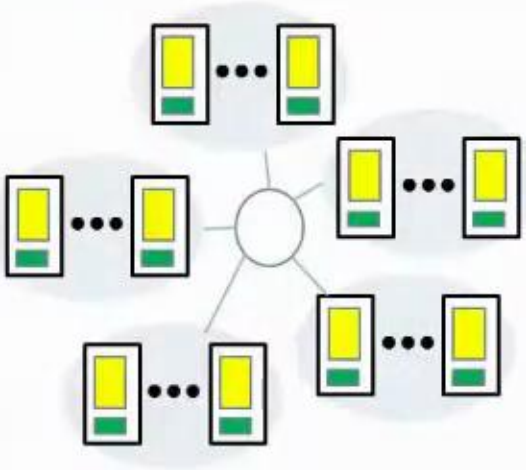
Cm*




One Kmap per cluster
50 compute modules (Cms)
5 communication controllers (Kmaps)
Cm 20
Cm 21
Cm 29
Kmap
Cm 10
Cm 11
Cm 19
Kmap
Kmap
Cm 30
Cm 31
Cm 39
Kmap
Kmap
Cm 00
Cm 01
Cm 09
Cm 40
Cm 41
Cm 49

Cm*





- Reliable message-passing
- NUMA
  - Same machine 3us
  - Same cluster 9us
  - Remote cluster 26us
- Contention and latency big issues when accessing remote memory
  - Sharing is expensive
  - Concurrent processes run better if independent

Medusa



- OS for Cm*, 1977-1980
- Goal: reflect the underlying distributed architecture of the hardware
- Single copy of the OS impractical
  - Huge difference in local vs non-local memory access times
- But complete replication of the OS also impractical
  - Small local memories (64 or 128KB)
  - Typical OS size 40-60KB

Medusa (cont)



- Replicated minimal kernel on each processor
  - Interrupts, context switching
- Other OS functions divided into disjoint "utilities"
  - Utility code always executed on its local processor
  - Utility functions invoked (asynchronously) by sending messages on pipes

Medusa (cont)



- OS utilities:
  - Memory manager
  - File system
  - Task force manager
    - All processes are task forces, consisting of multiple activities that are co-scheduled across multiple processors
  - Exception reporter
  - Debugger/tracer

Medusa (cont)



- Complicated to program
- For example, potential for deadlock when opening a file:
  - File manager must request storage for file control block from memory manager
  - If swapping between primary and secondary memory is required, then memory manager must request I/O transfer from file system

  → Deadlock
- Used co-scheduling of activities in a task force to avoid livelock
- Even the clock can't be trusted
  - Peripheral attached to one Cm*: subject to unpredictable contention and communication delays

Design issues



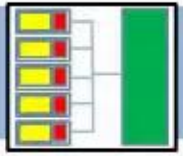
- Shared memory vs message passing
  - OS utility functions accessed only with message passing
  - Care taken to make message passing as efficient as possible
- Replication/caching
  - Limited replication of the OS (approx 1% of the code)
  - No caches
  - Execution of OS calls always local to data

Design issues (cont)



- Exploiting parallelism
  - Each activity in a task force statically assigned to a processor
  - Activities explicitly specify shared state, eg synchronization primitives
  - Activities can be dynamically added and removed for load balancing etc.
- Performance debugging
  - Debugger/Tracer utility keeps "performance measurements" for all utilities

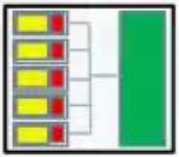
Firefly and Taos

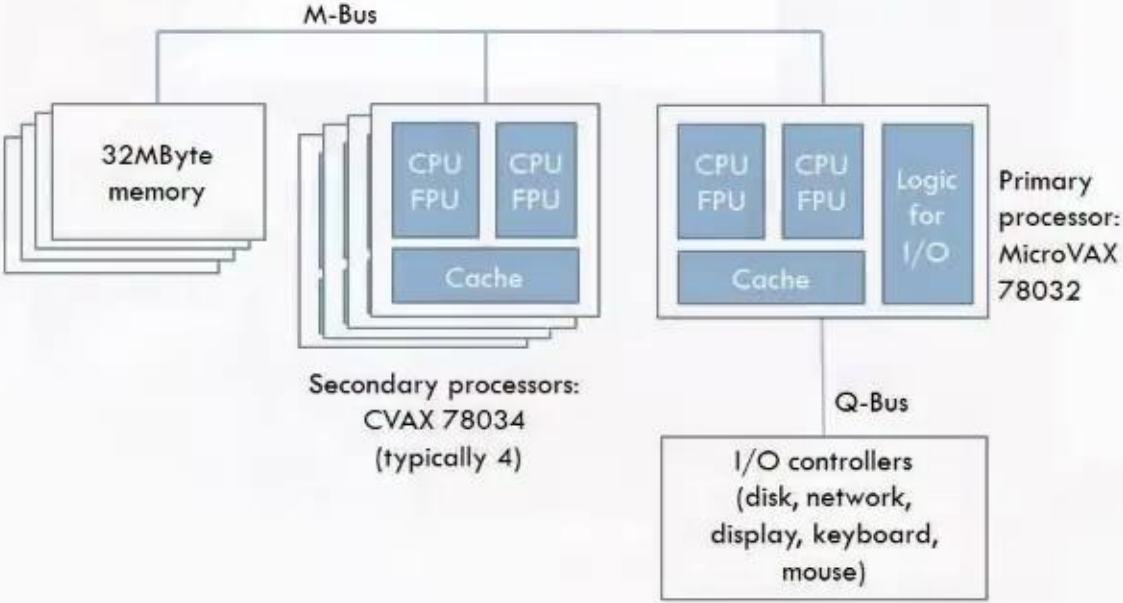


Firefly

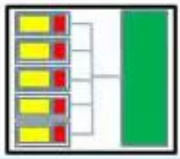
- Shared-memory, multiprocessor, personal workstation
  - Developed at DEC SRC, 1985-1987
- Requirements:
  - Research platform (powerful, multiprocessor)
  - Built in a short space of time (off-the-shelf components as much as possible)
  - Ease of programming (hardware cache coherence)

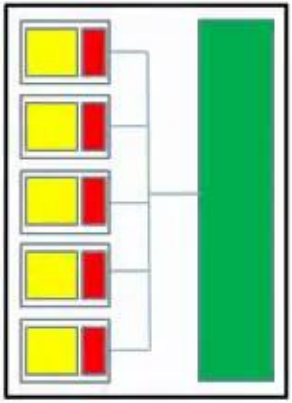
Firefly (version 2)





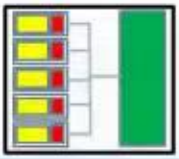
Firefly (cont)





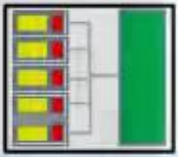
- UMA
- Reliable communications
- Hardware support for cache coherence
- Bus contention an important issue
  - Analysis using trace-driven simulation and a simple queuing model found that adding processors improved performance up to about 9 processors

Taos operating system



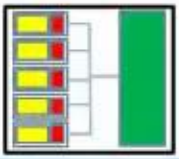
- Microkernel structure
- System kernel called the Nub
  - Virtual memory
  - Scheduler
  - Device drivers
  - Threads and synchronization
- Rest of the OS ran in user-mode as a distinguished process
  - File system, networking, etc.

Taos (cont)



- Supported multiple threads of control in a shared address space
  - Library support for programmers from the Topaz Threads module
- All software multithreaded
  - Executed simultaneously on multiple processors
- Uniform RPC communication mechanism
  - Same machine and between machines
  - Local RPC used to access OS functions in distinguished process

Design issues



- Shared memory vs message passing
  - Multiple threads in a single address space
  - Message passing (RPC) between processes
- Replication/caching
  - The first hardware cache coherent multiprocessor…
- Exploiting parallelism
  - Synchronization support between threads
- Performance debugging
  - Not high on the agenda

Aside: cache coherence protocols

Memory coherence

A memory system is coherent if

1. A read of location X by processor P will return the value previously written by P, if no other processor has written to X in between
   - it preserves program order
2. A read of location X by processor P2 will return the value previously written by P1, provided the read is "a sufficient time later" and there were no other writes in between
   - it gives a coherent view of memory
3. Writes to the same location are serialized
   - the order of writes is the same on all processors

Hennessy & Patterson, pp.205-207

Memory consistency

- Defines *when* a written value must be seen by a reader
- Sequential consistency
  - Writes by each processor are not reordered
  - Writes are serviced in the order they are received
  - Simple, but reduces potential performance
- If the program is assumed to be synchronized, more relaxed consistency models can be used for greater performance
  - The weaker consistency model is hidden from the programmer

Two styles of cache coherence

1. Write-through
   - Data updated both in the cache and in main memory
   - Simplifies coherency
   - Increased bus traffic

Two styles of cache coherence

2. Write-back
   - Data updated only in the cache
     - The block is marked as "dirty"
   - Written to main memory only when replaced in the cache (if dirty)
   - Low latency
   - Multiple writes to the same block only require one write to memory
   - Hard for multiprocessors

Hardware cache coherence

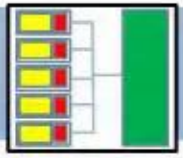
Example: **MOESI** protocol

- Every cache line is in one of five states:
  - **M**odified:dirty, present only in this cache
  - **O**wned: dirty, present in this cache and possibly others
  - **E**xclusive: clean, present only in this cache
  - **S**hared: present in this cache and possibly others
  - **I**nvalid: not present
- May satisfy read from any state
- Fetch to shared or exclusive state
- Write requires modified or exclusive state;
  - if shared, must invalidate other caches
- Owned: line may be transferred without flushing to memory

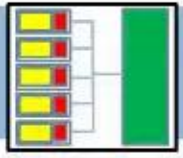
Snoopy protocols

- Every cache controller snoops requests on the bus
- Ties are broken by the bus arbitration scheme
  - If no bus, some other request serialization mechanism or a much more complicated state machine
- Lots of optimizations to minimize bus traffic generated by the coherence protocol and reduce access latencies

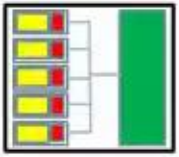
Firefly and Taos (cont)



Firefly and Taos (cont)

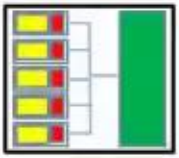


Cache coherence protocol



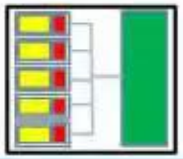
- Bus-based snoopy cache
- Conditional write-through
  - Lines can be writeable in multiple caches
  - For non-shared lines, use write-back
    - Dirty lines updated in main memory only when the line is victimized from the cache
  - For shared lines, use write-through
    - The cache must acquire the bus before it can modify the line
    - Update is visible to other caches

Cache coherence (cont)



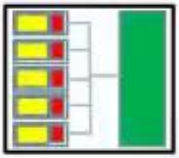
- Some disadvantages:
  - Write-through continues while datum is shared
    - Even if only one processor is actually modifying
    - **False sharing**
  - Process migration is expensive
    - Data will be held in two caches for a while
- Later snoopy protocols gave exclusive ownership to the writer of a block
  - Other copies are invalidated

Discussion: system balance



- Cache intended to reduce per-processor load on memory system
  - Rather than reduce average access time to main memory
- Hence low-performance bus able to support several processors
- In fact, two factors limited the processor count
  - Electrical constraints of the bus
  - Memory bandwidth
- This led to a "balanced" system
  - Low cache hit rates, but misses not very expensive
  - If processors were significantly faster compared to main memory, then hit rates would need to be increased with larger caches or cache block sizes

Discussion: RPC



- RPC used extensively, including between processes on the same machine
- Carefully tuned to be "fast enough", but didn't scale
  - At most 4,000 calls per second, and no increase with more processors
- Deliberately traded performance for
  - Simplicity: high-level language stubs
  - Correctness: isolated address spaces

Outline

- Introduction
  - Classification of computer systems
  - OS design issues
- A selection of multiprocessor and distributed OSes
  - C.mmp/Hydra
  - C*/Medusa
  - Firefly/Taos
  - Mach
  - Amoeba
  - VAXclusters
  - K42

Mach

Mach

- CMU, 1985-1994
- Goals included:
  - Support for both distributed computing and multiprocessing
  - Portability
  - Unix compatibility and comparable performance
- Key idea:
  - Same system abstractions whether closely- or loosely-coupled multiprocessor
  - Message-passing and virtual memory are duals
    - Each can be used in the implementation of the other
- Ported to many different hardware platforms
  - Supported as a product by several vendors

Structure

- Microkernel structure
  - Many kernel functions implemented as user-level servers
- Tasks (address spaces)
  - Multiple threads of control within a task
- Tasks mainly communicate via RPC (ie IPC)
  - Expensive compared to shared memory on tightly-coupled platform
  - Multiple software layers and some hardware boundaries for most task interactions irrespective of size of data transfer

IPC concepts

- Port
  - Communication channel (message queue)
  - Many senders, one receiver
  - Protected by the kernel, requires capability to access
  - All objects are represented by and manipulated through ports
- Message
  - Fixed length header + variable sized body
  - A single message can transfer up to the entire address space of a task
  - Messages are sent and received on ports
  - Can be blocking or nonblocking

Communication-VM duality

- Message-passing can be implemented using memory management primitives
  - Message contents are memory-mapped
  - Copy-on-write
    - Only physically copied if written
  - Efficient on tightly-coupled machine
- Memory management can be implemented using communications primitives
  - External user-level pager manages backing store
  - Since the kernel communicates with the pager using IPC, it can be on the same or a different machine
  - Flexible, same interface on loosely- and tightly-coupled machines

Design issues

- Shared memory ***and*** message passing
  - Modular structure
    - Ease of system design, implementation, maintenance
    - Failure isolation
    - Debugging, validation
    - Transparent access to network services
    - Flexibility
  - But finer granularity of protection mechanisms means greater cost of communication because domain boundaries have to be crossed every time

Performance of copy-on-write

- Depends on the message size!
  - Only efficient if enough data is involved
  - Gains from avoiding physical copying have to outweigh costs of housekeeping (page table manipulations)
- From Coulouris et al:
  - Milliseconds to copy messages of different sizes

size	Simple copy	Setup	Copy-on-write
8kb	1.4	1.57	4.82
256kb	44.8	1.81	66.4

Caching/replication

- Can get good locality for many clients via caches managed by user-space servers
- Intended from the outset to handle wide diversity of hardware
  - Can take advantage of hardware-provided cache coherence
  - Or can explicitly cache in the OS

Design issues: parallelism

- Three ways to achieve parallelism
  - A single "task" (process) with many threads in a shared address space
    - Library of synchronization primitives using shared memory
  - Multiple tasks with shared regions of address space
  - Multiple tasks communicating via messages
- Reflects the target multiprocessor architectures
  - UMA machines (eg VAX 11/784)
  - NUMA machines (eg BBN Butterfly)
  - Loosely-coupled networks of computers

Performance debugging in Mach

- Built-in kernel debugger
- Transparent forwarding of messages to remote nodes
  - Messages can be large, up to the size of a virtual address space
  - And copy-on-write semantics is potentially a complicating factor for performance debugging
- Not clear what facilities were available for monitoring IPC performance

Amoeba



Amoeba



- Vrije University, 1981-1996ish
- OS for a heterogeneous distributed system
- Intended to be a platform for research into distributed and parallel systems, languages, protocols and applications
- Design goals:
  - Distribution (many machines)
  - Parallelism (many CPUs)
  - Transparency (single system)
  - Performance

Aside: transparency

Definition

A distributed operating system is one that looks to its users like an ordinary centralized operating system but runs on multiple, independent central processing units.

"Distributed Operating Systems"
Andrew S. Tanenbaum and Robbert van Renesse
Computing Surveys, 17(4), December 1985

Definition

transparency

A distributed operating system is one that looks to its users like an ordinary centralized operating system but runs on multiple, independent central processing units.

"Distributed Operating Systems"
Andrew S. Tanenbaum and Robbert van Renesse
Computing Surveys, 17(4), December 1985

Transparency

- Location
  - of processes, files, hardware resources
- Replication
  - of files
- Concurrency
  - not notice other users
- Parallelism
  - auto parallelise serial programs

Amoeba (cont)



Hardware configuration



Workstations

Processor pool



Gateway

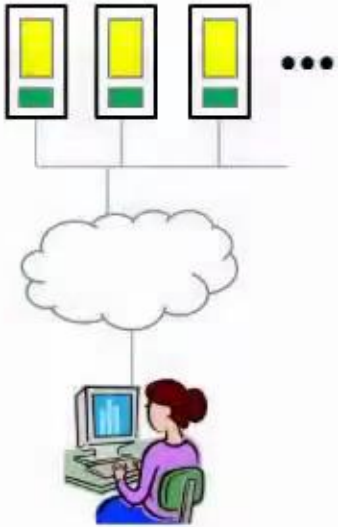
WAN

80 SPARC computers, each with private memory

Specialized servers (file, database, etc)

Amoeba

- Microkernel structure
  - Most OS services run as user-space servers, eg file, directory, run, time-of-day
- Object-oriented
  - Every object has a server process that manages it
- Capability-based protection
  - 128 bits, cryptographically protected



Communication and sharing



- IPC is blocking RPC
  - However threads in the same process communicate by shared memory
- Facility for reliable, totally-ordered group communication
  - Used by cooperating servers to update shared data
- User-level software (Orca) provides distributed shared objects

Location transparency



- Users are not aware of the location of servers or on which machines their programs are being executed
  - The "run" server selects processors from the pool based on free memory and CPU load
- RPC stubs include broadcast mechanism to locate server
  - Results are cached
  - But can take up to 30s
- Only transparent if the cache is hot and the necessary servers are up!



Design issues

- Shared memory vs message passing
  - All IPC is RPC
  - User-level software (Orca) provides a distributed, shared object overlay

Although it has long since been corrected, in Amoeba 2.0 we made a truly dreadful decision to have asynchronous RPC. In that system the sender transmitted a message to the receiver and then continued executing. When the reply came in, the sender was interrupted. This scheme allowed considerable parallelism, but it was impossible to program correctly. Our advice to future designers is to avoid asynchronous messages like the plague.



Design issues (cont)

- Caching/replication
  - Critical OS servers are potentially a bottleneck and single point of failure
    - Can be duplicated
    - Replicated data kept consistent transparently
- Exploiting parallelism
  - A process can not be split over multiple machines
  - Parallelism between processes rather than within
- Performance debugging
  - ?

Discussion: transparency



- Is transparency always desirable?
- Pros:
  - Flexible
  - Portable
  - Hide complexity
  - Modularisation
- Cons:
  - Unpredictable performance
- Pervasive challenge:
  - Provide transparency while at the same time take advantage of characteristics of the platform to get good performance

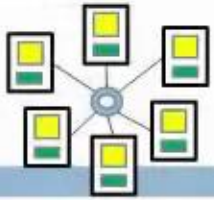
Outline

- Introduction
  - Classification of computer systems
  - OS design issues
- A selection of multiprocessor and distributed OSes
  - C.mmp/Hydra
  - C*/Medusa
  - Firefly/Taos
  - Mach
  - Amoeba
  - VAXclusters
  - K42

VAXclusters

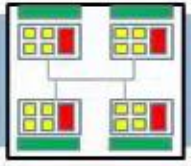


VAXclusters



- Mid 1980s to present
- Cluster of VAX computers running the VAX/VMS OS
  - General-purpose OS and commodity processor hardware
- Connected by high-speed (70mbs), message-oriented interconnect
  - Offload communications overhead to hardware
- "Closely-coupled" (vs tightly- or loosely-coupled)
  - Nodes run copies of the OS with own processor and memory
  - Needs high-speed, low-latency interconnect and shared storage
- Now called OpenVMS
  - Supports up to 96 nodes
  - Widely-used: more than 50,000 cluster installations

K42



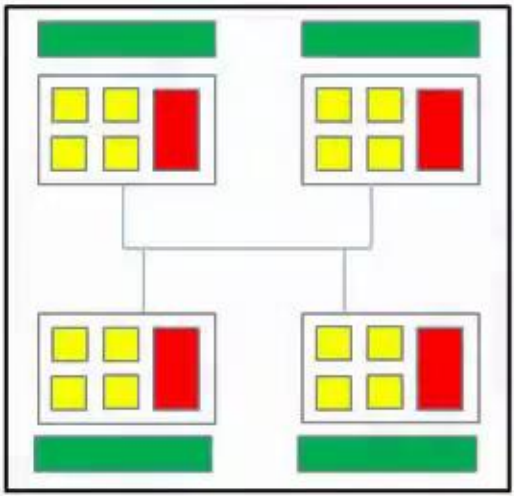
K42 multiprocessor OS

- IBM Research, 1997-2006ish
  - Successor of Tornado and Hurricane (University of Toronto)
- Targets large-scale ccNUMA
- Goals:
  - High locality, scalability
  - Reliability, fault tolerance
  - Scale to hundreds of processors
  - Customizable and portable
    - Able to exploit architecture-specific features for performance

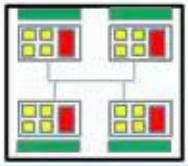


ccNUMA

- Reliable message passing
- Hardware cache coherence
  - Deep cache hierarchy
- Shared memory relatively expensive to access

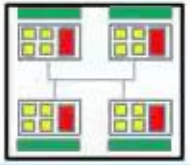


K42 features



- Modular, object-oriented
  - Resources managed by a set of object instances
  - Avoids global data structures and locks
- Microkernel
  - Linux API and ABI support
  - Provides memory and process management, IPC, scheduling, networking, device drivers
- Open-source project

Why use OO in an OS?

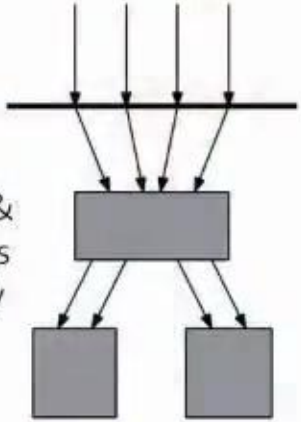


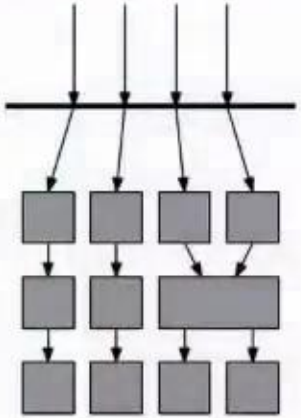
Traditional System

OO Decomposed System

User-level requests

System paths & data structures used to satisfy requests

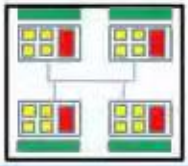




- much sharing

- much **less** sharing
- better performance

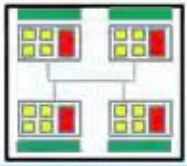
Clustered Objects



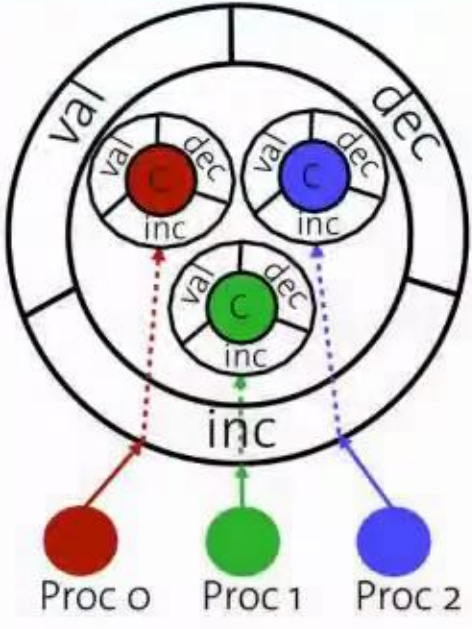
- Objects encapsulate resources, including the locks required to manipulate that resource
  - Allows for high spatial and temporal locality
- Some objects are inevitably shared across multiple processors
  - Reduce sharing contention by implementing as clustered objects
- Clustered objects are transparently partitioned or replicated across the multiprocessor
  - Same reference on any processor; invocations are routed to one of the object representatives (reps)

⇒ Choice of sharing and locking strategy local to each object

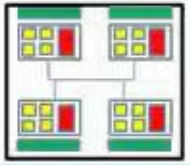
Example: clustered object for shared counter



- If *inc* and *dec* are the common operations then most efficient if *inc* and *dec* are local; only *val* needs to communicate
- If val is the common operation then a shared implementation of val will perform better

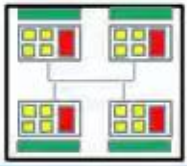


Using clustered objects



- Degree of clustering (number of reps, partitioned vs replicated) depends on how the object is used
- State maintained by the object reps must be kept consistent
- Determining global state is potentially costly
  - Eg How to choose the next highest priority thread for scheduling when priorities are distributed across many user-level scheduler objects

Design issues



- Message passing vs shared memory
  - Support for both, but implementation mostly shared memory
- Replication/caching
  - A clustered object manages its own replication internally using a library of lock-free data structures
- Exploiting parallelism
  - Localised OS resources so OS doesn't limit parallelism of the application
- Performance debugging
  - Built-in fine-grained, online, lightweight tracing
  - Visualisation tools

Summary

- This talk presented a historical perspective on multiprocessor operating systems and distributed operating systems
- Main take-home point:
  - At any scale, distributed systems designers need to be cognizant of the relative costs of operations
- Next lecture: what is the relevance of all this today? Do we need a new approach to operating systems on multicore hardware?